PROJECT REPORT: Cloud-Native Monitoring & Autoscaling of 'FitMotion website' on Kubernetes

Student Name: Parnav Kumar

Registration Number = 12209660

Roll Number = 41

Section = 4OM05

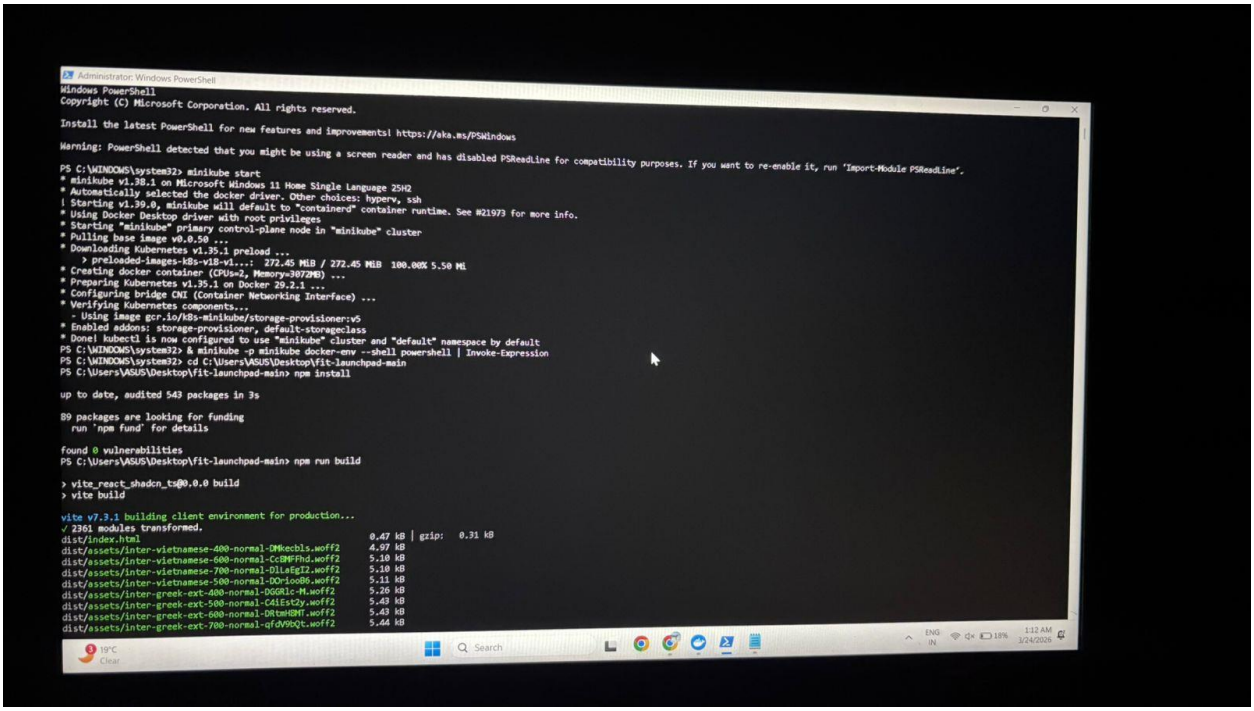
---

1. Project Overview

This project demonstrates a complete DevOps lifecycle: containerizing a web application (**FitMotion**), deploying it to a managed Kubernetes environment (**Minikube**), and implementing a professional-grade observability stack using **Prometheus** and **Grafana**. The final phase includes stress-testing the application to validate **Horizontal Pod Autoscaling (HPA)** based on real-time metrics.

2. Environment Initialization

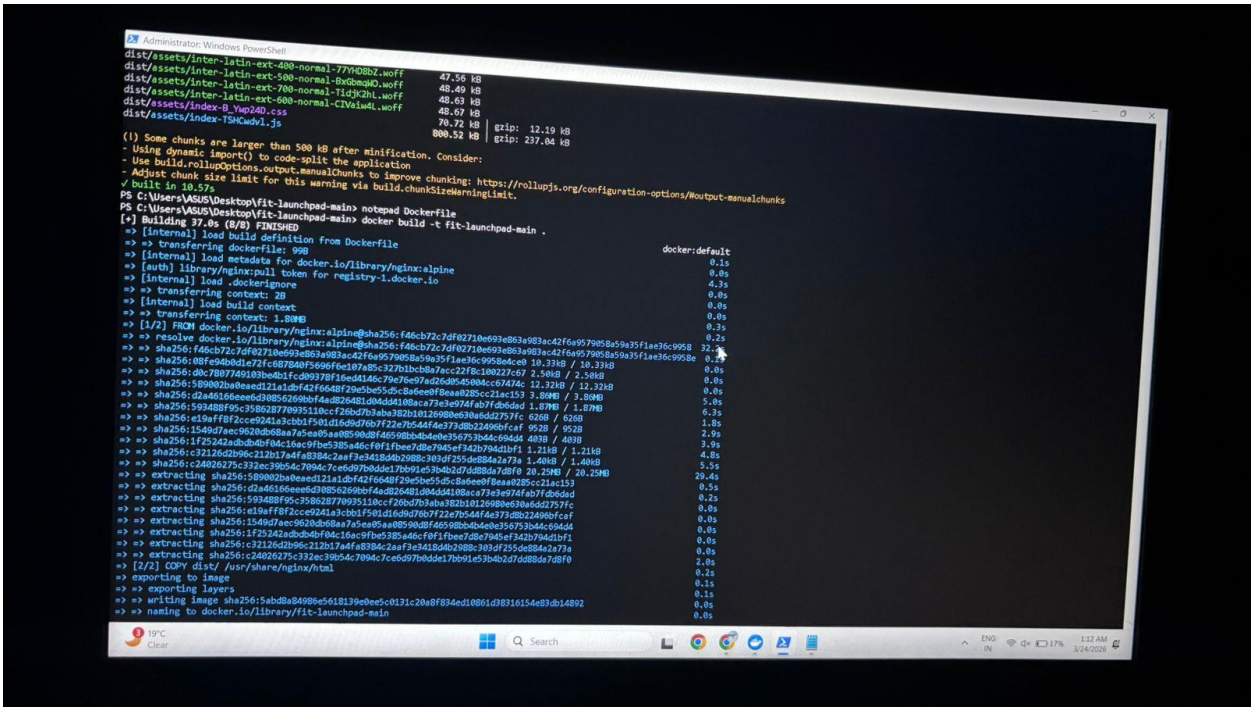
The project began by spinning up a local Kubernetes cluster using Minikube on Windows 11. This provides the necessary control plane and worker node infrastructure.



3. Application Build & Deployment

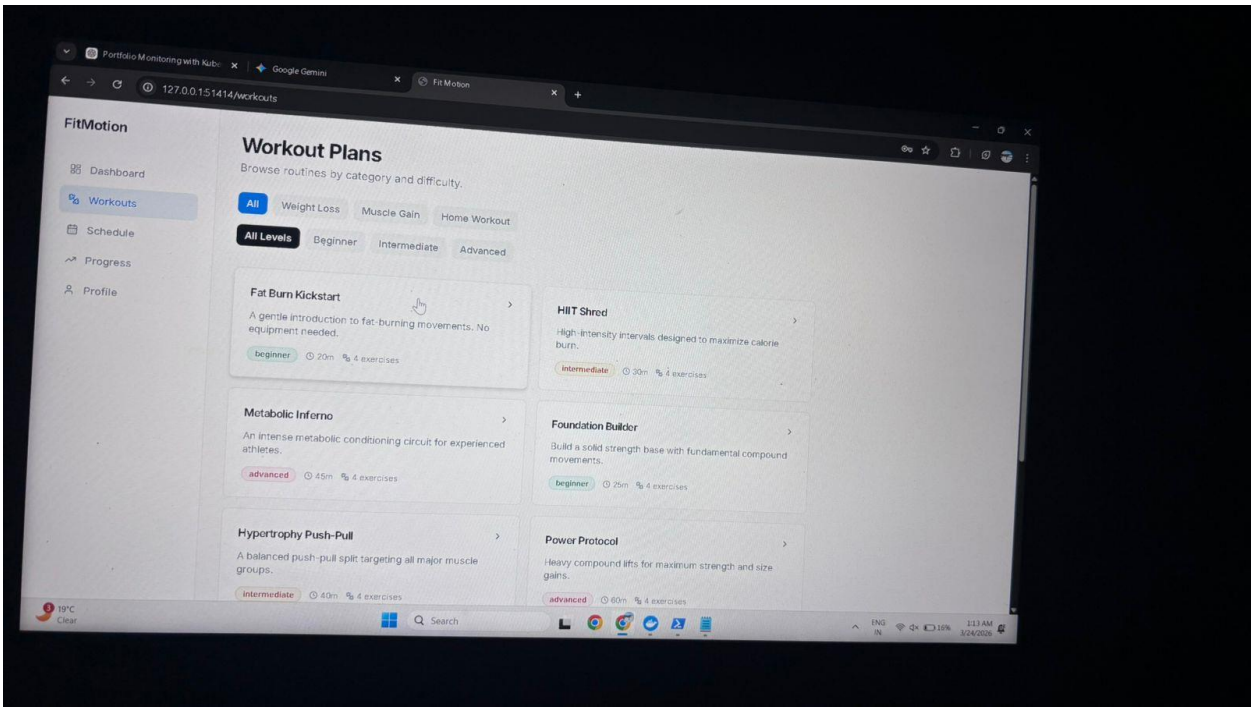
The 'FitMotion' application (a React-based workout platform) was built from source. A multi-stage Dockerfile was used to compile the assets and serve them via Nginx.

3.1 Docker Build Process



3.2 Application Verification

Once the deployment and service were applied, port-forwarding was used to ensure the application was reachable by users.



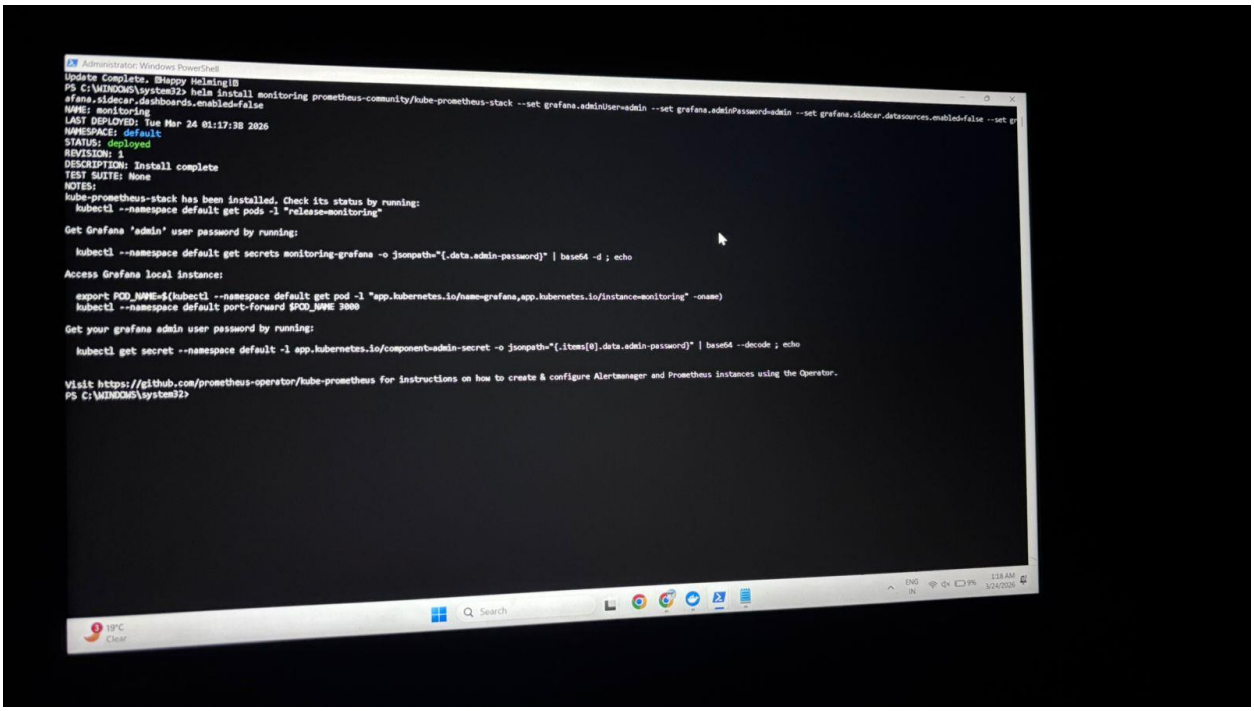
---

4. Implementing the Monitoring Stack

To achieve observability, the **Kube-Prometheus-Stack** was deployed via Helm. This bundle includes the Prometheus Operator, Grafana, and several exporters.

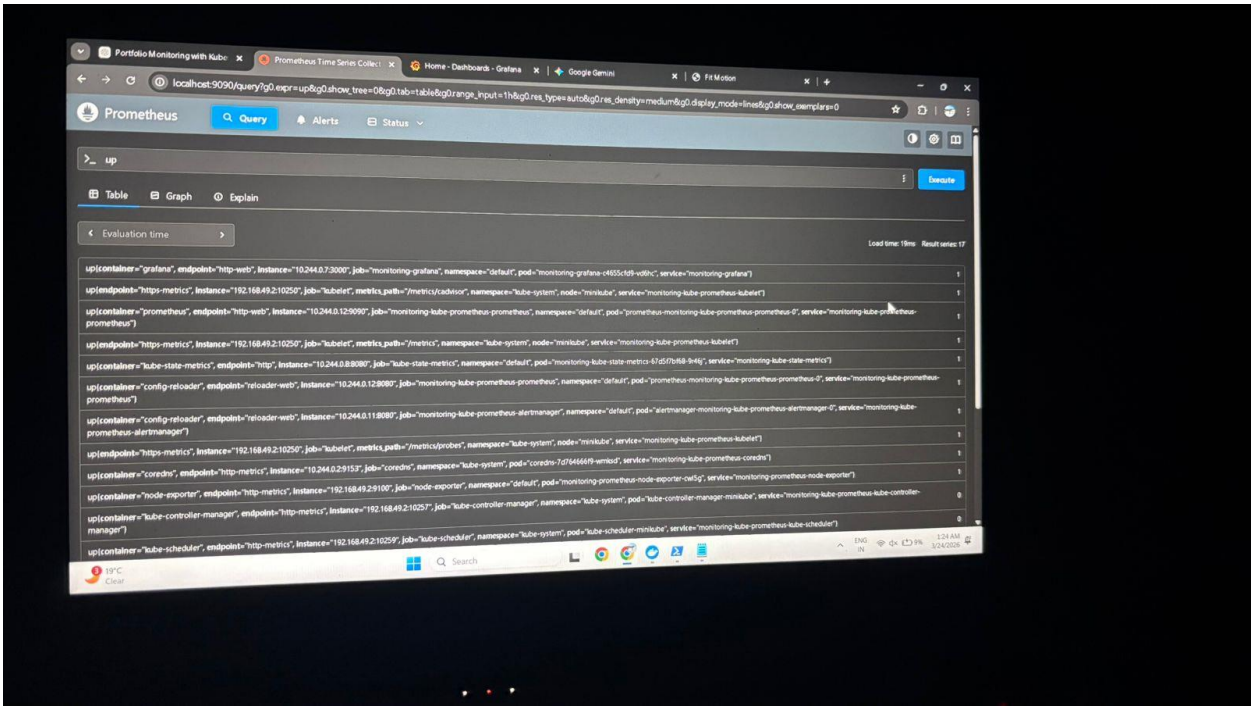
4.1 Helm Installation

The stack was configured with custom administrative credentials to ensure secure access to the dashboards.



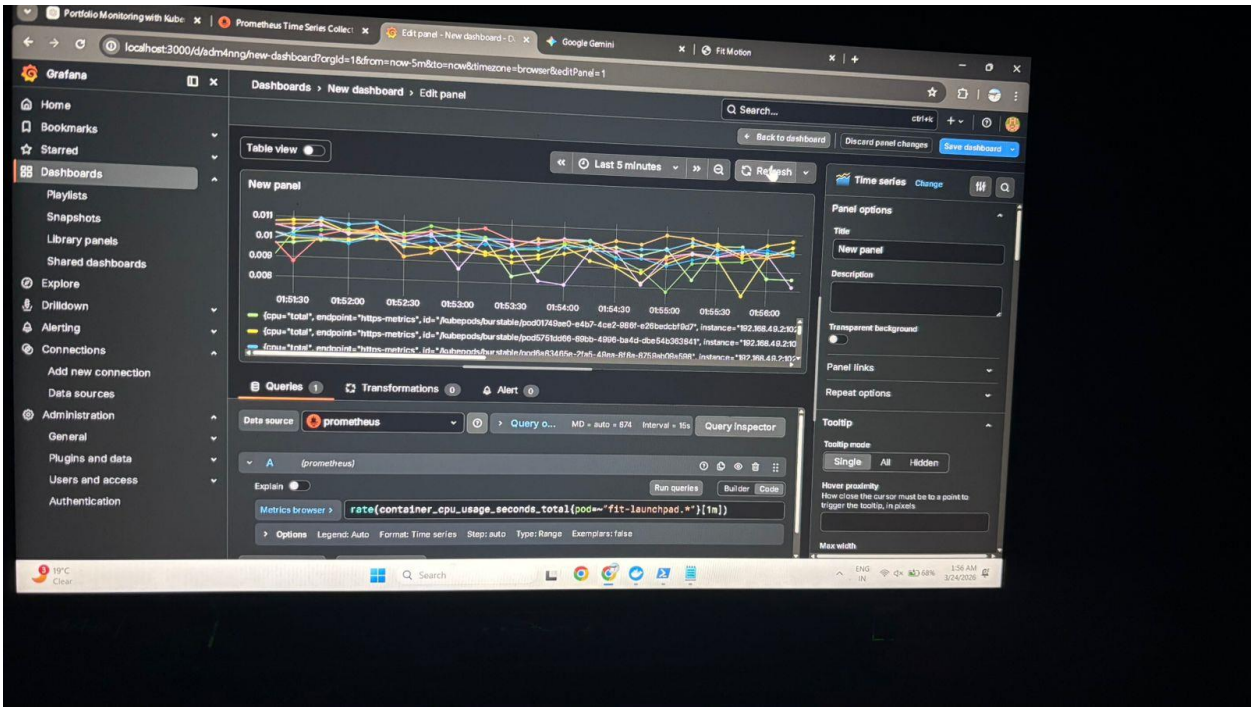
4.2 Backend Verification (Prometheus)

Before visualizing data, the Prometheus target list was checked to ensure all Kubernetes components (Kubelet, Node Exporter, etc.) were being "scraped" correctly.



4.3 Visualization (Grafana)

A custom dashboard was created to track the CPU usage of the fit-launchpad pods specifically, using PromQL queries.



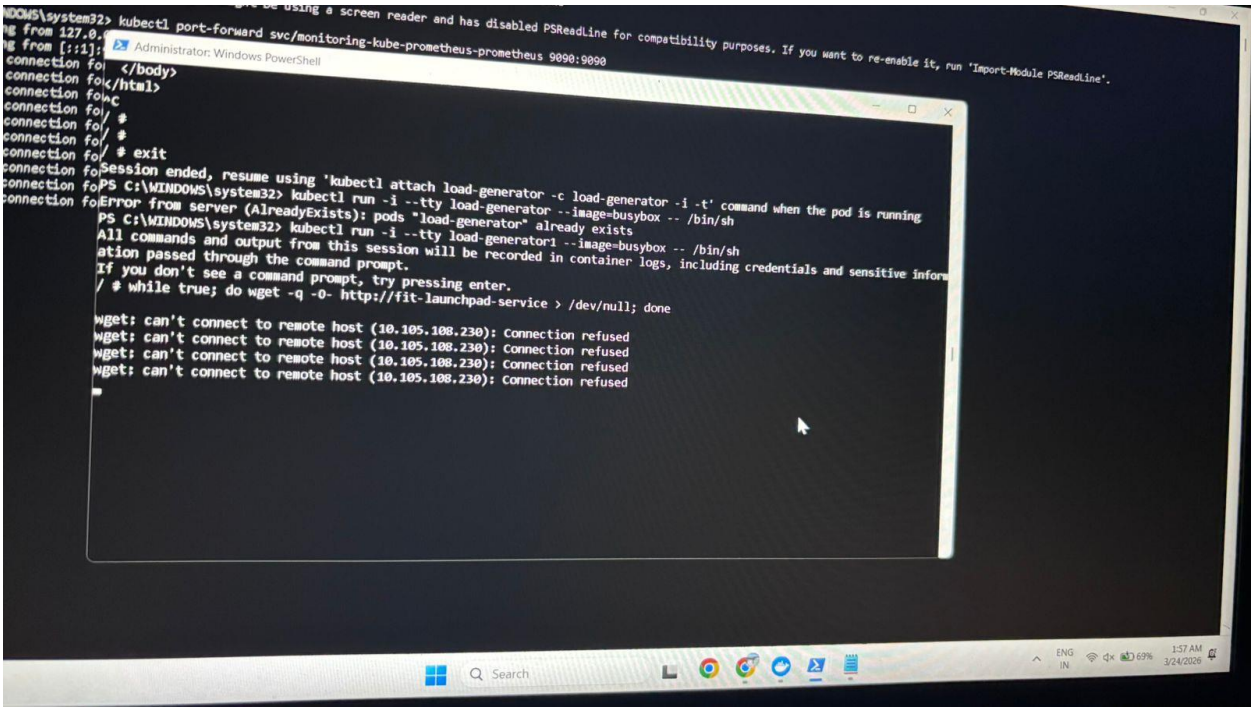
---

5. Load Testing & Troubleshooting

To test the "DevOps" capabilities of the cluster, a load generator was deployed to simulate high traffic.

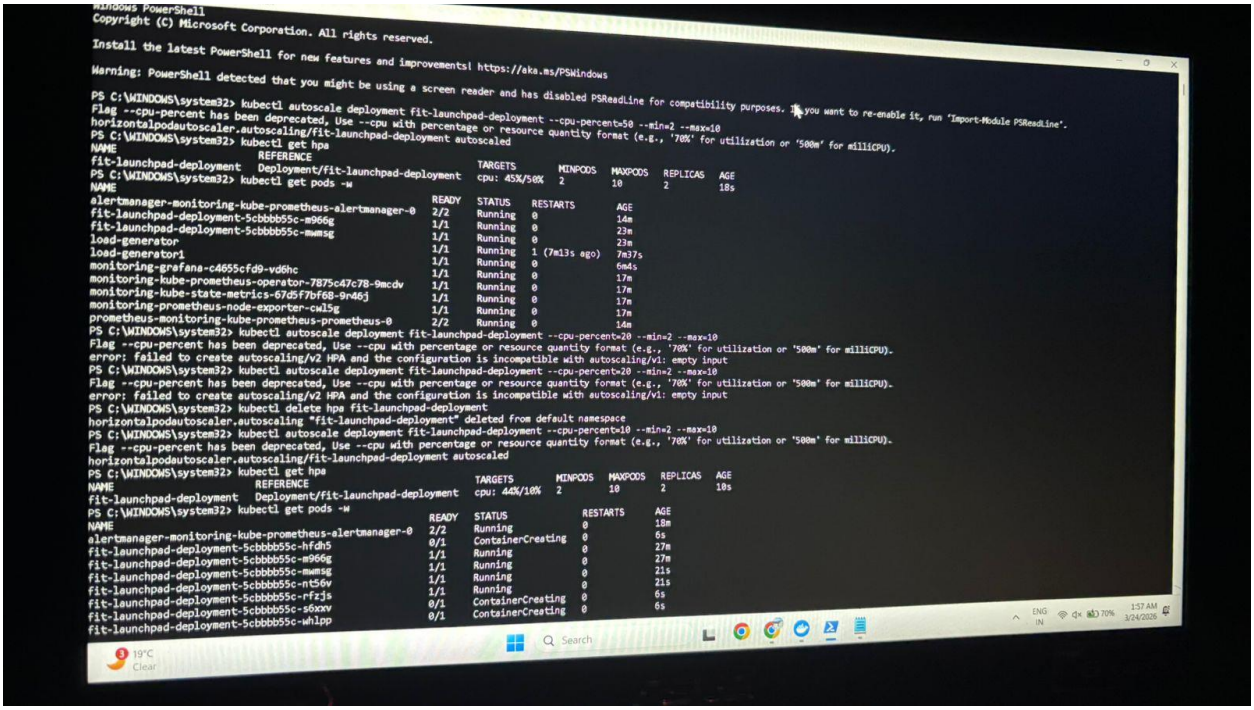
5.1 Connectivity Testing

Initial attempts to hit the service resulted in connection errors, requiring network troubleshooting within the cluster.



5.2 HPA Configuration Errors

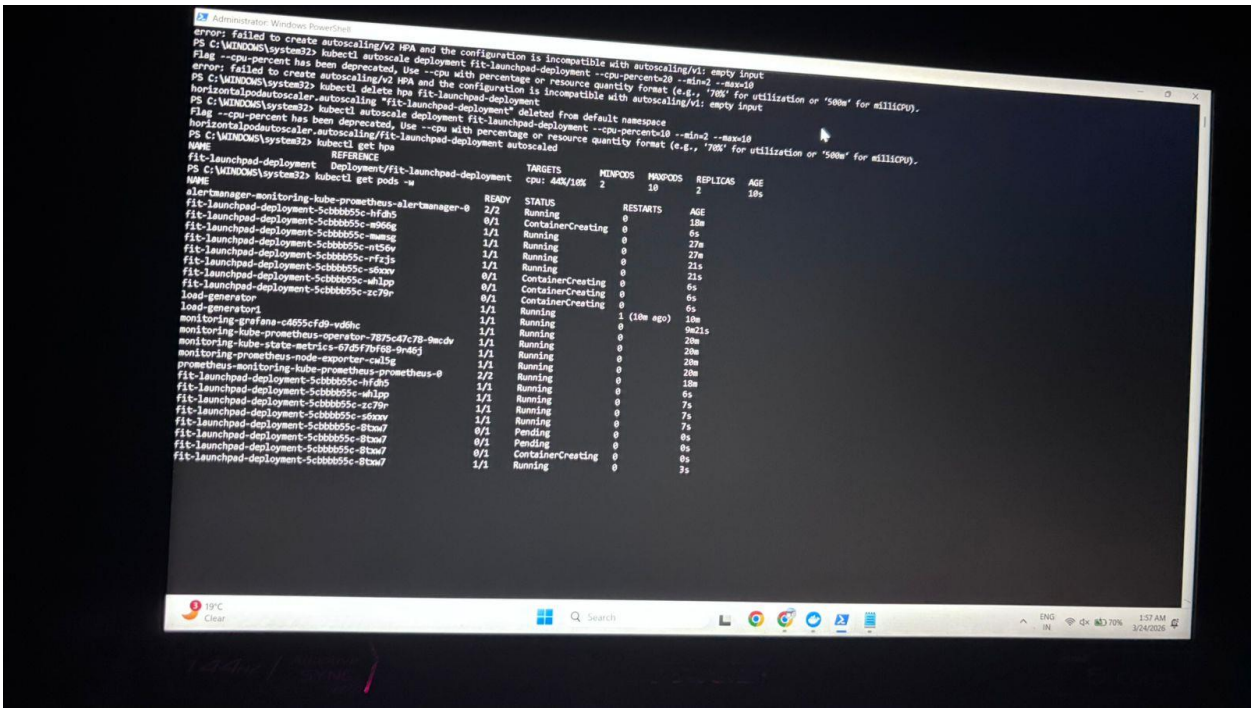
Attempts to enable autoscaling via the CLI faced versioning conflicts and deprecated flag errors, which were resolved by adjusting the autoscaling/v2 parameters.



[INSERT img 10.jpeg HERE] > *Figure 8: Troubleshooting PowerShell errors related to HPA resource quantity formats and API versions.*

6. Final Result: Successful Scaling

After resolving configuration issues, the Horizontal Pod Autoscaler was successfully initialized. Under the simulated load, the cluster detected the rise in CPU usage and automatically scaled the deployment from 2 replicas to 10.



7. Conclusion

This project successfully bridged the gap between development and operations. By implementing a full monitoring suite, we transformed a "black box" deployment into a transparent, self-healing system. This setup ensures that as the 'FitMotion' user base grows, the infrastructure will automatically scale to meet demand while providing the DevOps team with real-time performance data.